pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

APIs, Keys, and Secrets

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From files to an API

Courses 1 and 2 read every number from files in the repo. Course 3 also asks a server: FRED, through the FRED API. Every request carries your API key.

Your key never goes into code, a notebook, a saved output, a slide, a video, or a commit.

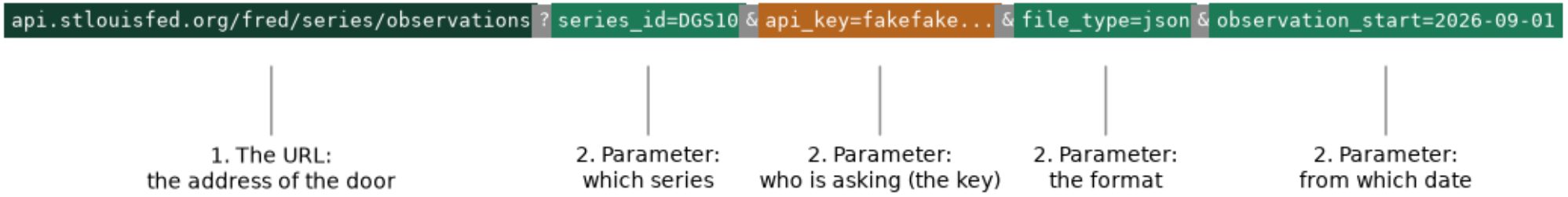
No key yet? Every cell still runs. Live cells print a skip message, and that is what the saved notebook shows.

Week 1: getting the data in

Day 1	**APIs, keys, and secrets**
Day 2	JSON and the FRED API
Day 3	Market calendars, cleaning
Day 4	Resampling, the as-of value
Day 5	Case study: the dataset

What an API is: four parts

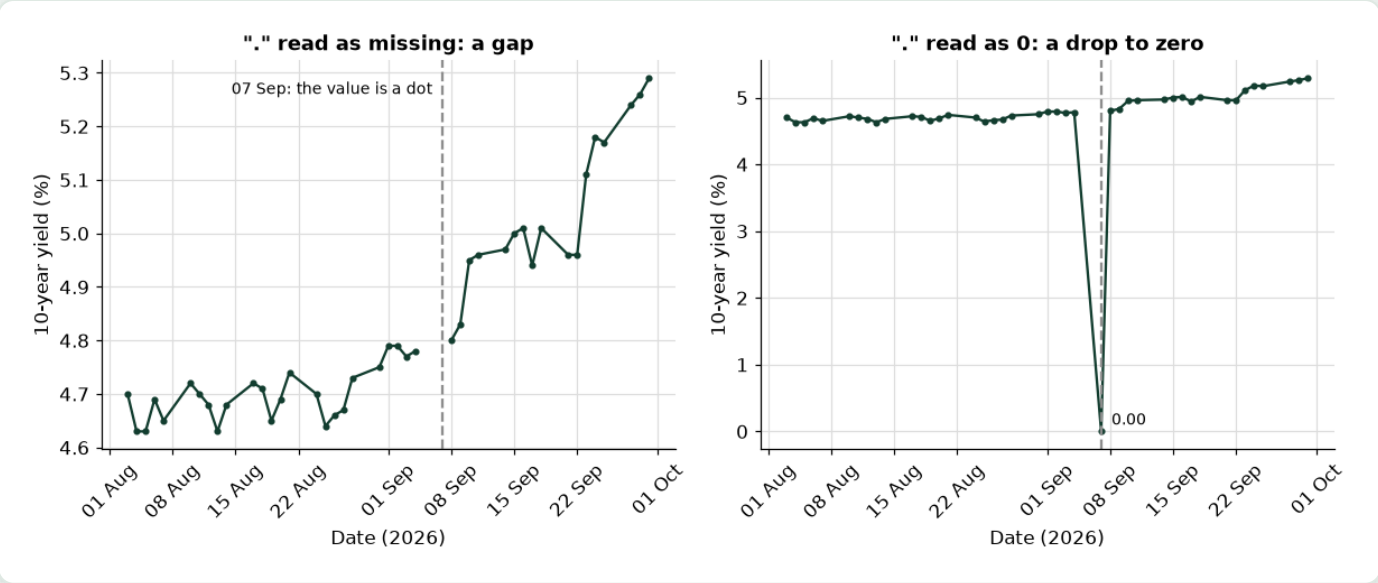
One FRED request, split into its parts



Sent back: 3. a status code (200 means success), and 4. the body (the answer, as JSON text)

The key travels inside the URL. Print a URL, log it, or put it in an error, and the key goes with it.

What comes back: a body and a status



Request	Status
Everything right	200
A well-formed key FRED does not know	400
A key of the wrong shape	400
A series that does not exist	400
More than 120 requests in a minute	429

Keep the gap, never the zero. 400 means your request was wrong; 429 means too many in a minute.

Getting a FRED key

1. Sign in

Your FRED account, at the apikeys page of fredaccount.stlouisfed.org. Keys need a login.

2. Request

Request an API key on that page and follow its steps.

3. Copy, do not paste

32 lowercase letters and digits. Section 1.10 gives it its place.

"All users of an application shall use their own API key." FRED API documentation

A colleague who wants to run your notebook registers for their own key. You never send yours.

A key is a password

A password	An API key
Proves who you are at log in	Proves who you are on every request
Anyone with it can act as you	Anyone with it can send requests as you
Never written in a shared place	Never in code, a notebook, a slide, a commit
Changed at once when it leaks	Replaced at once when it leaks

Your key, not your firm's. At work, follow your firm's policy. In Colab, public or synthetic data only.

Environment variables

```
os.environ["DEMO_API_KEY"] = FAKE_KEY
os.environ["DEMO_API_KEY"] == FAKE_KEY   # True
# compare, never print
del os.environ["DEMO_API_KEY"]
```

Named text values around every program. Code asks for the key by name and never contains it. A program started from the session inherits a copy.

PowerShell (shown, not run)

`$env:DEMO_API_KEY = "..."`

This terminal window only

`[Environment]::SetEnvironmentVariable(...)`

Saved for your Windows user

PowerShell saves what you type in its history file. A real key goes in a file you edit instead.

The .env file and .env.example

.env: the values. Never in Git.

```
# today's drill: the fake key
DEMO_API_KEY=fakefake...fake
```

.env.example: names only. In Git.

```
# the secrets this folder needs
DEMO_API_KEY=
```

```
values = dotenv_values(work / ".env")   # a dictionary
"DEMO_API_KEY" in os.environ            # False: the environment is untouched
```

dotenv_values with an explicit path: nothing is copied into the environment, and the temp folder search problem goes away.

.gitignore before the first commit

```
# secrets
.env
# downloads and databases
cache/
*.db
__pycache__/
.pytest_cache/
```

```
git status --short --ignored
!! .env
git check-ignore -v .env
.gitignore:2:.env
```

.gitignore stops Git starting to track a file. A file already committed stays tracked, and stays in history.

The order: .gitignore, check it with git check-ignore -v, and only then the first commit.

get_secret: three places, in order

1. Environment

`os.environ.get(name)`

→

2. Colab's panel

`only when google.colab imports`

→

3. The .env file

`dotenv_values(env_file)`

Found nowhere: a RuntimeError that names the secret and the places, never a value or a folder.

```
RuntimeError: Secret NO_SUCH_SECRET not found. Looked in: the environment variable NO_SUCH_SECRET, Colab's Secrets panel, and the file .env given as env_file.
```

Four tests with monkeypatch and tmp_path: environment, file, environment wins, missing. 4 passed.

Colab's Secrets panel

1. Open

The key icon in the bar on the left: Secrets.

2. Add

Name FRED_API_KEY, your key as the value.

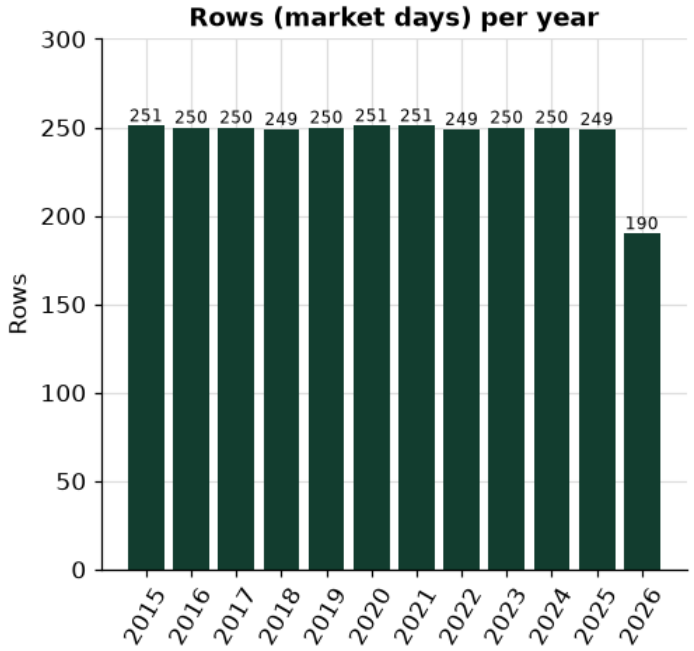
3. Allow

Turn on notebook access for this notebook.

The secret stays in your Colab account, not in the notebook. Sharing the notebook does not share the key.

The snapshot behind every analysis





2,940 rows, 2015-01-02 to 2026-10-02. Only market days have a row, about 250 a year; the last bar is short because the file stops there.

The key cell and a live cell

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_response = requests.get(FRED_URL, ...)
    # the status code, never the URL
    ...checked against the Treasury file
```

A live cell, two ways

No key (the saved notebook):

`Live cell skipped: no FRED key was found.`

With your key:

status 200, ten rows, every date equal

Never type fred_key alone in a cell, never print it, never print a URL.

ICE BofA and Moody's series: their terms forbid publishing, so their values never appear in a recording or a saved notebook (day 5).

Your real key: my_bondmath\.env

1	**.env.example**	`Set-Content .env.example "FRED_API_KEY="`
2	**.gitignore first**	`Add-Content .gitignore ".env", "cache/", "*.db"`
3	**Edit, not type**	`code .env, then paste after FRED_API_KEY=`
4	**Check**	`git check-ignore -v .env`
5	**Rerun**	`Restart the kernel: the key cell says found`

In Colab: the Secrets panel. Never upload a .env file to Colab.

When a key leaks: rotate first

A commit, a pushed repo, a shared notebook output, a screenshot, a recording, a chat with a colleague or an AI assistant.

Rotate

New key, into .env; stop using the old one.

Deleting is not enough

Git keeps every commit; the old one still holds it.

History rewrite

Exists, slow, misses copies. Named, not done.

GitHub Docs: "rotate the affected credential immediately"; removing it from history is "often unnecessary if you've already revoked the credential."

GitHub Docs, About secret scanning; Removing sensitive data from a repository (read 2026-10-04).

Excel side by side

A URL from cells

```
="...?series_id="&A1
&"&api_key="&B1
&"&...&observation_start="
&ENCODEURL(C1)
```

=ENCODEURL("a b&c=d/e?f") returns a%20b%26c%3Dd%2Fe%3Ff

A key in a cell

Fake key in B1, column B hidden, saved as .xlsx: still in the file (xl/sharedStrings.xml) and back in B1 on reopening.

Every copy of the workbook carries it.

From Web, no key

Power Query on fredgraph.csv?id=DGS10: header observation_date, DGS10; 16,893 rows from 1962 on the day tested.

WEBSERVICE on it: #VALUE!. Not used.

Tested in Excel 16.0 build 20430, 2026-10-04, with the fake key only.

Save the day: the leak drill

1. Commit the key

settings.py with the fake key, committed

2. Ignore it

still in git ls-files

3. Delete it

git rm, commit: gone from the folder

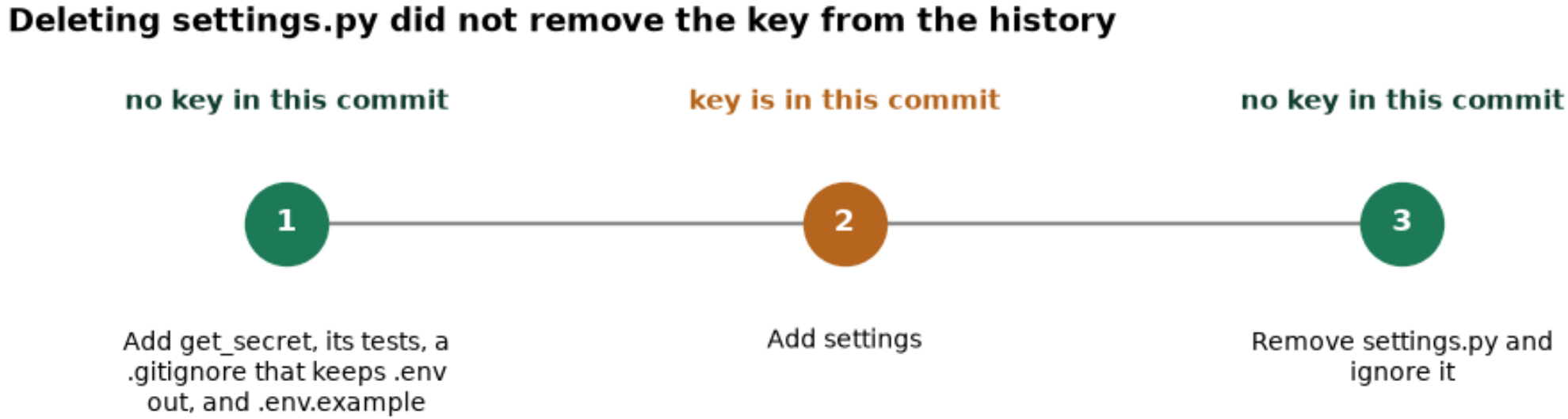
4. Search

git log -S finds both commits; git show HEAD~1 prints the key

```
git show HEAD~1:settings.py    ->    DEMO_API_KEY = "fakefakefakefakefakefakefakefake"
```

The fix: rotate. A new fake key (...rotated1) in .env, ignored, so nothing to commit.

Where the key stays



Git never edits a commit. Deleting a file adds a commit without it; the old commit keeps the key. Rotate first.

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: describe_request promises a log line safe to print, and builds it with api_key= and the key.

Code that runs is not the same as code that is right.

What you can do now

- Name the four parts of an API request; read a status code
- Keep a key in .env or an environment variable, and never print it
- .gitignore before the first commit, proved with git check-ignore -v
- get_secret: environment, Colab, file, tested with monkeypatch and tmp_path
- Run live cells with or without a key; rotate a leaked key first

Week 1, Day 2 JSON and the FRED API: the body of the answer, the missing-value dot, and a dated series